



Restricción de Rutas, Página de Login Personalizada y Manejo de Errores 403 en Spring Security

Objetivo

Restringir accesos según el rol del usuario, personalizar la página de **Login**, mostrar un **Error 403** amigable y capturar el **usuario logueado**.

◆ Paso 1: Modificar la clase `SecurityConfig.java`

Ubicación:

`src/main/java/gm/sga/web/SecurityConfig.java`

<https://www.globalmentoring.com.mx>

Agrega el siguiente método para configurar la autorización:

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(authorizeRequests -> authorizeRequests
            .requestMatchers("/login", "/resources/**").permitAll() // Permitir acceso a login y recursos
            // estáticos
            .requestMatchers("/editar/**", "/agregar/**", "/eliminar/**").hasRole("ADMIN") // Restricción
            // de acceso por rol
            .requestMatchers("/").hasAnyRole("USER", "ADMIN") // Acceso a usuarios autenticados
            .anyRequest().authenticated() // Cualquier otra solicitud requiere autenticación
        )
        .formLogin(formLogin -> formLogin
            .loginPage("/login") // Página de login personalizada
            .defaultSuccessUrl("/", true) // Redirigir después del login exitoso
            .permitAll() // Permitir acceso al formulario de login
        )
        .logout(logout -> logout
            .logoutUrl("/logout")
            .logoutSuccessUrl("/login?logout")
            .invalidateHttpSession(true)
            .deleteCookies("JSESSIONID")
        )
        .exceptionHandling(exceptionHandling -> exceptionHandling
            .accessDeniedPage("/errores/403")
        );
    return http.build();
}
```

✓ Explicación rápida:

- Solo usuarios ADMIN pueden acceder a editar, agregar y eliminar.
- Usuarios USER y ADMIN pueden acceder al inicio (/).
- Se define una **página de login personalizada**.
- Se configura una **página de error 403** para accesos denegados.

◆ Paso 2: Configurar la clase `WebConfig.java`

Ubicación:

`src/main/java/gm/sga/web/WebConfig.java`

Agrega o modifica el método para mapear las vistas:

```
@Override
public void addViewControllers(ViewControllerRegistry registry) {
    registry.addViewController("/").setViewName("index");
    registry.addViewController("/login").setViewName("login");
    registry.addViewController("/errores/403").setViewName("/errores/403");
}
```

✓ Explicación:

- / → Vista index.html.
- /login → Vista personalizada login.html.
- /errores/403 → Vista personalizada de error 403.

◆ Paso 3: Crear la página login.html

Ubicación:

src/main/resources/templates/login.html

Contenido:

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Login</title>
    <meta charset="UTF-8"/>
  </head>
  <body>
    <h1>Login</h1>

    <form method="POST" th:action="@{/login}">
      <label for="username">Username:</label>
      <input type="text" name="username" id="username"/>
      <br/>
      <label for="password">Password:</label>
      <input type="password" name="password" id="password"/>
      <br/>
      <input type="submit" value="Login"/>
    </form>
```

```
</body>
</html>
```

✓ Recuerda que `name="username"` y `name="password"` son **obligatorios** para que Spring Security los reconozca.

◆ Paso 4: Crear la página de error `403.html`

Ubicación (Debemos crear la carpeta de errores dentro de templates):

```
src/main/resources/templates/errores/403.html
```

Contenido:

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Acceso Denegado</title>
    <meta charset="UTF-8"/>
  </head>
  <body>
    <div>Acceso denegado.
      NO tienes permisos para ver esta página o ejecutar esta acción
      <a th:href="@{/}" >[[#{accion.regresar}]]</a>
    </div>
  </body>
</html>
```

✓ Esta página se mostrará si un usuario intenta acceder a algo sin permisos.

◆ Paso 5: Capturar el usuario que hizo login (en `ControladorInicio.java`)

Ubicación:

```
src/main/java/gm/sga/controlador/ControladorInicio.java
```

Agrega en el método principal el parámetro:

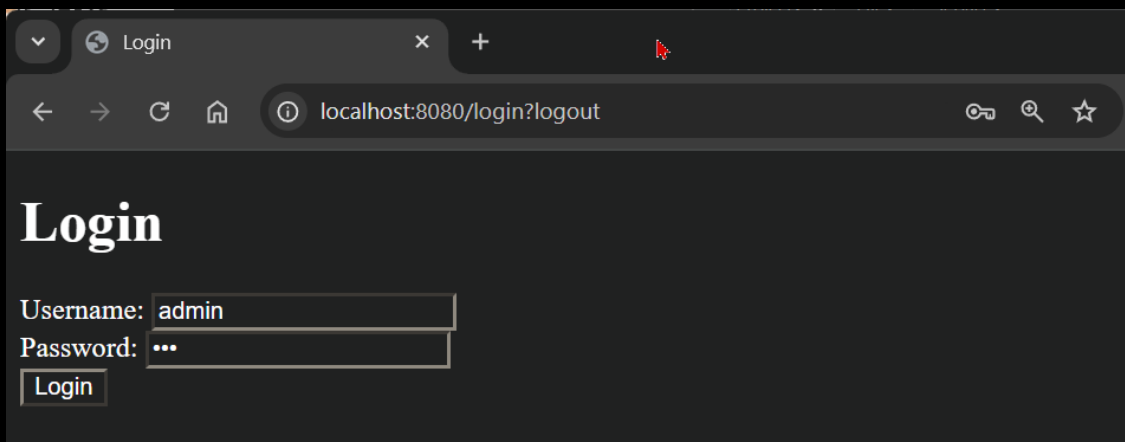
```
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.core.userdetails.User;

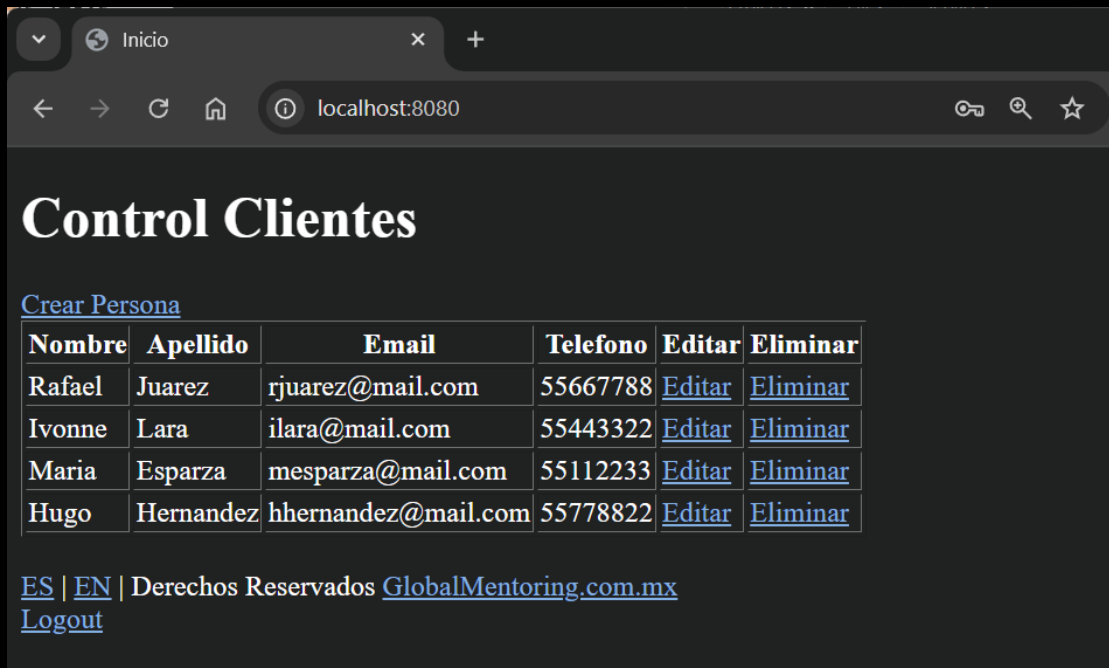
@GetMapping("/")
public String inicio(Model model, @AuthenticationPrincipal User user) {
    log.info("Ejecutando el controlador Spring MVC");
    log.info("usuario que hizo login:" + user);
    var personas = personaService.listarPersonas();
    model.addAttribute("personas", personas);
    return "index";
}
```

✓ Ahora puedes ver en la **consola** quién hizo login cada vez que se accede a la página principal.

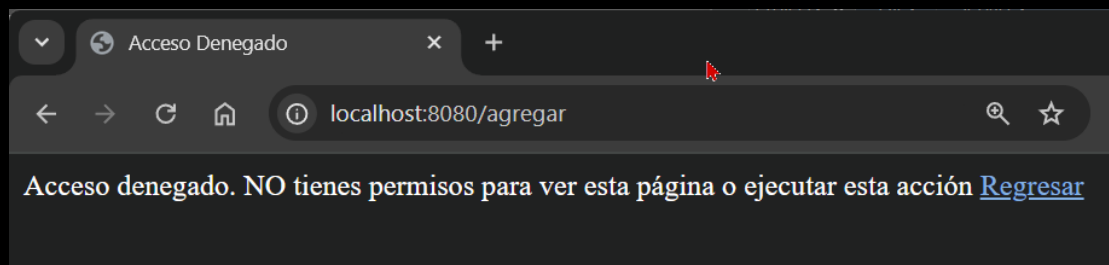
◆ Paso 6: Ejecutar y Probar

1. Ejecuta tu proyecto (Run).
2. Abre <http://localhost:8080>.
3. Inicia sesión con:
 - Usuario: `admin`
 - Password: `123`
-
- Usuario: `user`
- Password: `123`
4. Verás que puedes realizar todas las acciones con el usuario **admin**, pero solo podrás listar con el usuario **user**.
5. Haz clic en **Logout** para cerrar sesión para cambiar de usuarios y probar sus roles asignados.





Si entramos con el usuario **user** e intentamos realizar una acción no permitida, como editar un registro, veremos la siguiente pantalla:



✨ Código Final Consolidado

SecurityConfig.java:

```
package gm.web;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;
```

```

@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(authorizeRequests -> authorizeRequests
                .requestMatchers("/login", "/resources/**").permitAll() // Permitir acceso a login y
recursos estáticos
                .requestMatchers("/editar/**", "/agregar/**", "/eliminar/**").hasRole("ADMIN") //
Restricción de acceso por rol
                .requestMatchers("/").hasAnyRole("USER", "ADMIN") // Acceso a usuarios autenticados
                .anyRequest().authenticated() // Cualquier otra solicitud requiere autenticación
            )
            .formLogin(formLogin -> formLogin
                .loginPage("/login") // Página de login personalizada
                .defaultSuccessUrl("/", true) // Redirigir después del login exitoso
                .permitAll() // Permitir acceso al formulario de login
            )
            .logout(logout -> logout
                .logoutUrl("/logout")
                .logoutSuccessUrl("/login?logout")
                .invalidateHttpSession(true)
                .deleteCookies("JSESSIONID")
            )
            .exceptionHandling(exceptionHandling -> exceptionHandling
                .accessDeniedPage("/errores/403")
            );
        return http.build();
    }

    @Bean
    public UserDetailsService userDetailsService() {
        return new InMemoryUserDetailsManager(
            User.withUsername("admin")
                .password("{noop}123")
                .roles("ADMIN", "USER")
                .build(),
            User.withUsername("user")
                .password("{noop}123")
                .roles("USER")
                .build()
        );
    }
}

```

```
}
```

WebConfig.java:

```
package gm.web;

import java.util.Locale;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.LocaleResolver;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.ViewControllerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import org.springframework.web.servlet.i18n.LocaleChangeInterceptor;
import org.springframework.web.servlet.i18n.SessionLocaleResolver;

// Indicamos que esta clase es de configuración
@Configuration
public class WebConfig implements WebMvcConfigurer {

    // Definimos el Bean para resolver el idioma de la sesión
    @Bean
    public LocaleResolver localeResolver() {
        var slr = new SessionLocaleResolver();
        // Establecemos el idioma por defecto (Español)
        slr.setDefaultLocale(Locale.forLanguageTag("es"));
        return slr;
    }

    // Definimos el Bean para permitir el cambio de idioma mediante un parámetro
    @Bean
    public LocaleChangeInterceptor localeChangeInterceptor() {
        var lci = new LocaleChangeInterceptor();
        // Indicamos que el parámetro para cambiar de idioma será "lang"
        lci.setParamName("lang");
        return lci;
    }

    // Registramos el interceptor para que Spring MVC detecte cambios de idioma
    @Override
    public void addInterceptors(InterceptorRegistry registro) {
        registro.addInterceptor(localeChangeInterceptor());
    }
}
```



```
@Override
public void addViewControllers(ViewControllerRegistry registry) {
    registry.addViewController("/").setViewName("index");
    registry.addViewController("/login").setViewName("login");
    registry.addViewController("/errores/403").setViewName("/errores/403");
}
}
```

ControladorInicio.java:

```
package gm.web;

import gm.domain.Persona;
import gm.servicio.PersonaService;
import jakarta.validation.Valid;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.core.userdetails.User;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.Errors;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class ControladorInicio {

    private static final Logger log = LoggerFactory.getLogger(ControladorInicio.class);

    @Autowired
    private PersonaService personaService;

    @GetMapping("/")
    public String inicio(Model model, @AuthenticationPrincipal User user) {
        log.info("Ejecutando el controlador Spring MVC");
        log.info("usuario que hizo login:" + user);
        var personas = personaService.listarPersonas();
        model.addAttribute("personas", personas);
        return "index";
    }

    @GetMapping("/agregar")
```

```
public String agregar(Persona persona) {
    return "modificar";
}

@PostMapping("/guardar")
public String guardar(@Valid Persona persona, Errors errores) {
    if (errores.hasErrors()) {
        return "modificar";
    }
    personaService.guardar(persona);
    return "redirect:/";
}

@GetMapping("/editar/{idPersona}")
public String editar(Persona persona, Model model) {
    persona = personaService.encontrarPersona(persona);
    model.addAttribute("persona", persona);
    return "modificar";
}

@GetMapping("/eliminar")
public String eliminar(Persona persona) {
    personaService.eliminar(persona);
    return "redirect:/";
}
}
```



Ahora:

- Tu app restringe páginas según el rol.
- Tienes una página de Login personalizada.
- Muestras un error 403 personalizado.
- Recuperas el usuario logueado fácilmente.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)